



Networking and Volumes in Docker

Essential Infrastructure for Stateful AI Applications

Why Networking & Volumes Matter

Container Isolation

Containers are **isolated by default**, creating secure but disconnected environments

Connection Needs

Networking connects containers to users, services, and each other

Data Persistence

Volumes persist data beyond container lifecycle, essential for stateful applications

Together they enable **scalable, stateful AI applications** - core skills for deploying ML in production

Docker Networking Basics

- Private IPs

Each container gets its own private IP address

- Default Bridge

Default network is **bridge**, enabling basic communication

- Network Access

Containers can communicate if on the same network



Host maps container ports to public ports with:

```
docker run -p 8000:8000 ai-app
```

Port Mapping

`-p host_port:container_port` syntax exposes internal services to the outside world

Critical for user-facing components like APIs, dashboards, and Jupyter notebooks

Example: `-p 8888:8888` exposes Jupyter to browser access

Essential for inference endpoints that need external requests

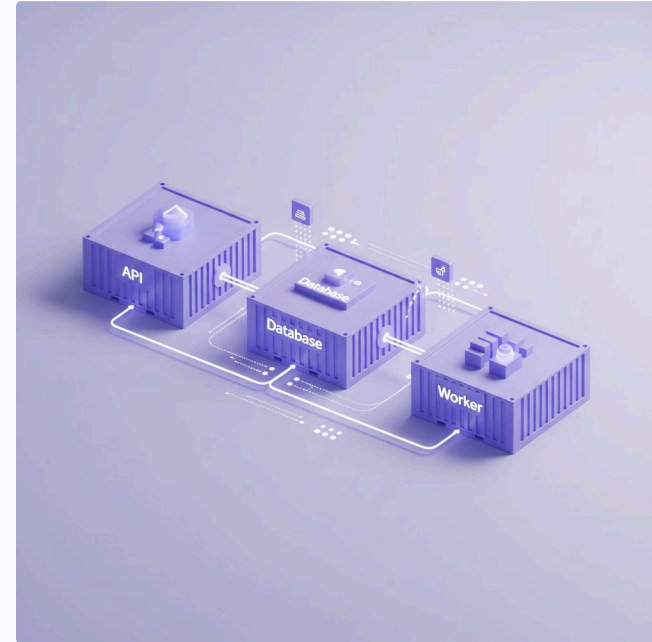
Without mapping, services are only accessible internally

Custom Docker Networks

```
docker network create ai-net  
docker run -d --name db --network ai-net postgres  
docker run -d --name app --network ai-net ai-app
```

Containers on the same custom network can:

- Communicate using container **names as hostnames** (e.g., db:5432)
- Maintain isolation from other containers
- Establish secure multi-service AI pipelines



Perfect for complex ML systems with databases, APIs, and worker containers

Docker Volumes Basics

⚠ The Ephemeral Container Problem

Containers are temporary by design. Without volumes, all data is lost when containers are deleted or restarted!

How Volumes Solve This:

- Persist data on host system
- Mount with special syntax:

```
docker run -v /host/data:/container/data ai-app
```

Maps host path to container path



Volumes keep your **datasets, trained models, and logs safe** - critical for ML workflows

Named Volumes

1

Create Named Volume

```
docker volume create model-  
store
```

Creates a managed volume with
Docker

2

Use in Container

```
docker run -v model-  
store:/app/models ai-app
```

Mounts volume at specified
container path

3

Share Across Containers

```
docker run -v model-  
store:/models inference-api
```

Same volume used in multiple
containers

Named volumes are [perfect for shared datasets or model checkpoints](#), preventing duplicated data and improving portability

Bind Mounts

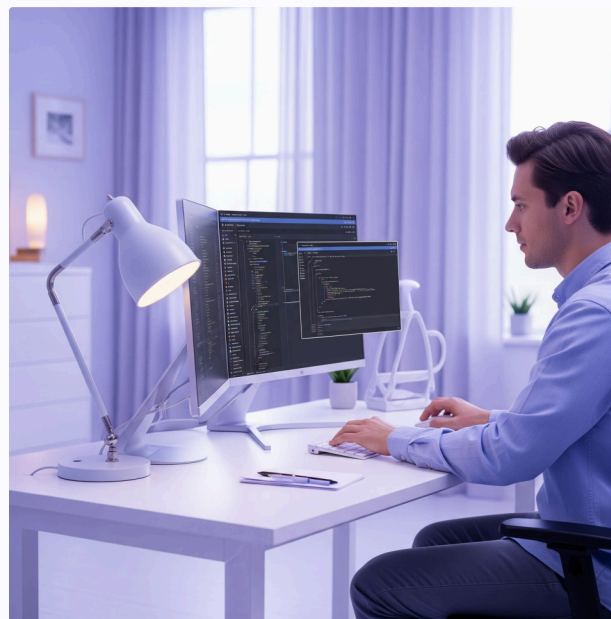
Developer-Friendly Mounts

Map host directory directly to container:

```
docker run -v $(pwd)/notebooks:/workspace jupyter
```

Perfect for development workflows:

- Syncs local code changes instantly
- Edit with your favorite IDE
- Container immediately sees changes
- No rebuild needed for code tweaks



Risk Alert

Bind mounts can overwrite container files! Use carefully in production environments.

Using Volumes for AI Models



Store Large Models

Keep pre-trained models (often several GB) on dedicated volumes



Share Across Containers

Multiple inference containers access same model files



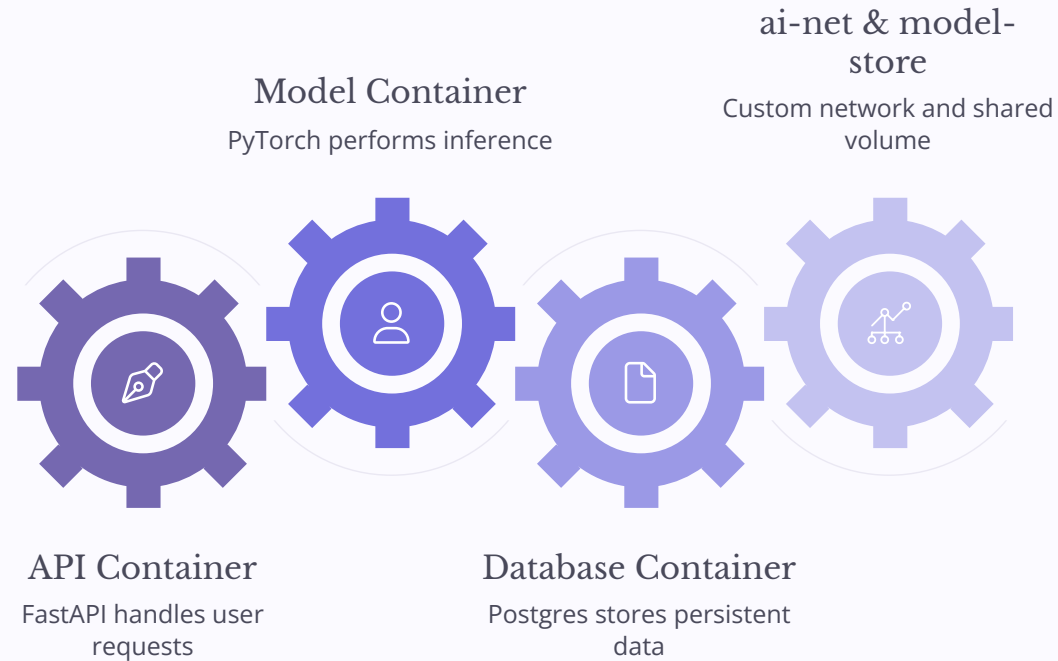
Optimize Performance

Volume acts as "model cache" - no repeated downloads

This approach [saves bandwidth](#), [reduces costs](#), and [boosts speed](#) in production environments



Combining Networks + Volumes



This architecture creates [modular, maintainable AI systems](#) where:

- API container handles user requests via port mapping
- Database container stores persistent application data
- Model container performs inference using shared models
- All components communicate securely on private network

Key Takeaways



Networking

Connects containers to each other and users



Volumes

Persist datasets, logs, and trained models

Together they form the [foundation for multi-container orchestration](#) with Kubernetes

Mastering Docker networking and volumes is a **must-have skill** for production AI infrastructure engineers

Next Steps:

- Practice creating custom networks
- Implement volume strategies for your ML workflows
- Explore Docker Compose for multi-container setups